

UNIVERSIDADE DE COIMBRA



FCTUC FACULDADE DE CIÊNCIAS
E TECNOLOGIA

MACHINE LEARNING

PREDICTIVE MODELING FOR DIABETES RISK ASSESSMENT

MAY 2025

BRUNO AZEVEDO



GONALO CARTAXANA



Table of Contents

Introduction	3
Background	4
Problem Definition	5
Exploratory Data Analysis	6
Dataset Structure	6
Data Dimensions	6
Data Types Overview	6
Binary Features	6
Continuous and Ordinal Features	7
Visualization: Violin Plot	7
Visualization: Histogram of Binary Features	7
Target Variable Distribution	8
Initial Imbalance Observations	8
Class Imbalance Assessment	8
Outlier Detection	9
Boxplot Analysis of BMI	9
Correlation Analysis	10
Input Feature Correlation	10
Correlation with Target Variables	10
Class-Specific Correlation in Ternary Target	11
One-Hot Encoded Feature Correlation with Ternary Target	11
Principal Component Analysis (PCA)	12
Data Preprocessing	13
Data Cleaning	13
Duplicate Removal	13
Outlier and Missing Data Handling	13
Splitting into Scenarios	13
Data Splitting	13
Feature Engineering	14
One-Hot Encoding (OHE) — Not Applied	14
Box-Cox Transformation (Normalization)	14
Standardization (StandardScaler)	14
Dimensionality Reduction — Not Applied	14
Methodologies	15
Random Forest	15
Parameters:	15
Chosen grid search parameters:	15
Support Vector Machines	16
Parameters:	16
Chosen grid search parameters:	16
Multilayer Perceptron	17

Parameters:	17
Chosen grid search parameters:	17
Results and Discussion	18
Results	18
Analyzing the Binary Scenario	20
Impact of Preprocessing	20
Influence of Data Sampling Techniques	20
Undersampling	20
Hybrid Sampling	20
Comparative Model Performance	20
Analyzing the Ternary Scenario	21
Trade-offs Between Metrics	21
Cross-Scenario Pattern Analysis	21
Optimizing Grid Search - SVM Execution Time	21
Conclusions	22
References	23

Introduction

This report presents a comprehensive analysis of diabetes risk prediction using data from the 2015 Behavioral Risk Factor Surveillance System (BRFSS) [1]. Diabetes, a prevalent chronic disease, affected approximately 34.2 million Americans as of 2018. It is characterised by the body's inability to regulate blood glucose levels effectively, potentially leading to severe complications such as cardiovascular disease, kidney failure, and vision impairment [2]. Additionally, an estimated 88 million Americans are living with prediabetes, and the economic burden of undiagnosed diabetes and prediabetes is projected to approach \$400 billion annually. Early identification of at-risk individuals is therefore critical for effective public health intervention.

This study aims to develop predictive models for diabetes risk by analysing the BRFSS 2015 dataset, which includes responses from 441,455 individuals across 330 health-related variables. The significance of this work lies in its potential to inform targeted prevention strategies, reduce the disease burden, and improve population health outcomes through data-driven insights.

The report is organised into the following sections: background, exploratory data analysis, methodologies, results and discussion, and conclusion, each contributing to a comprehensive understanding of the project's scope and impact.

Background

The application of machine learning techniques to diabetes risk prediction has gained significant attention due to the rising prevalence of the disease and the availability of large-scale health datasets such as the Behavioral Risk Factor Surveillance System (BRFSS). Predictive modeling enables the early identification of at-risk individuals, allowing for timely interventions that can reduce complications and healthcare costs.

Several studies have explored the use of machine learning algorithms, particularly Random Forest (RF), Support Vector Machines (SVM), and Multilayer Perceptron (MLP), for diabetes prediction. Choi et al. [3] applied RF, SVM, and neural networks to the 2014 BRFSS dataset, achieving area under the receiver operating characteristic curve (AUC-ROC) values ranging from 0.7182 to 0.7949. While their neural network attained the highest accuracy (82.4%), its low sensitivity (39.8%) highlighted the challenge of detecting minority diabetic cases. Their use of the Synthetic Minority Oversampling Technique (SMOTE) informs this project's strategy for addressing class imbalance.

Sadhu and Jadli [4] reported a 98% accuracy using RF on the UCI diabetes dataset, but their study did not address missing data, an important issue when working with BRFSS. In contrast, this project incorporates imputation and outlier handling to ensure data integrity. Zhang et al. [5] developed an RF-based model using community health data, achieving 91.24% accuracy and an AUC of 0.97. However, their reliance on clinical biomarkers, which are absent in BRFSS, underscores the complexity of working with self-reported survey data.

Farhad et al. [6] applied RF and MLP to a five-year cohort study, with MLP achieving an F1-score of 82.97% after using SMOTE, reinforcing the value of resampling in handling imbalanced datasets. Nguyen et al. [7], in a systematic review of machine learning models for type 2 diabetes, noted that RF performs well (up to 99% accuracy) on small datasets but identified difficulties when scaling to large and complex datasets like BRFSS.

Collectively, these studies underscore the effectiveness of RF, SVM, and MLP in diabetes prediction, while also highlighting persistent challenges such as handling missing data, class imbalance, and high-dimensional features. This project builds on their findings by applying these algorithms to the 2015 BRFSS dataset, integrating advanced preprocessing and feature selection to improve predictive performance and support data-driven public health strategies.

Problem Definition

This study addresses the task of diabetes diagnosis classification using self-reported health data from the 2015 BRFSS dataset. Two classification scenarios are considered:

- **Binary classification:** distinguishing between individuals without diabetes (class 0) and those with diabetes (class 1).
- **Ternary classification:** expanding the target classes to include individuals with prediabetes, resulting in three categories: no diabetes (class 0), prediabetes (class 1), and diabetes (class 2).

Each classification scenario uses the same set of 21 input variables in addition to the target variable. These features encompass a range of demographic, behavioral, and health-related factors, such as age, BMI, physical activity, smoking status, alcohol consumption, general health, and healthcare access. This diverse feature set supports a robust analysis of risk factors contributing to diabetes.

The choice of classification scheme affects model complexity, evaluation, and interpretability, and is intended to explore different levels of diagnostic granularity in public health applications.

Exploratory Data Analysis

Exploratory Data Analysis (EDA) was conducted to derive initial insights into the dataset and strategically inform subsequent methodological decisions. This phase involved the application of various visualization techniques to elucidate underlying patterns and characteristics within the data.

Dataset Structure

The datasets used for binary and ternary classification share the same set of 21 input features. The only difference lies in their respective target variables: **Diabetes_binary** for the binary scenario and **Diabetes_012** for the ternary scenario. For the purposes of exploratory data analysis, both target variables are included to provide a unified understanding of the feature distributions and their relationship to diabetes outcomes.

Data Dimensions

The dataset contains 253,680 observations and 23 columns in total: 21 input features and 2 target variables. This structure is confirmed by inspecting the data using **head()** and **shape**, which outputs: **(253680, 23)**.

Data Types Overview

All features in the dataset are of float type, indicating that the data consists exclusively of numerical values. There are no textual or categorical string variables present, simplifying preprocessing and model input preparation.

Binary Features

A total of 14 features are binary, representing mostly yes/no responses to survey questions. These include:

['HighBP', 'HighChol', 'CholCheck', 'Smoker', 'Stroke', 'HeartDiseaseorAttack', 'PhysActivity', 'Fruits', 'Veggies', 'HvyAlcoholConsump', 'AnyHealthcare', 'NoDocbcCost', 'DiffWalk', 'Sex'].

These features typically take values 0 or 1, corresponding to negative or affirmative responses.

Continuous and Ordinal Features

The remaining 7 features are continuous or ordinal in nature:

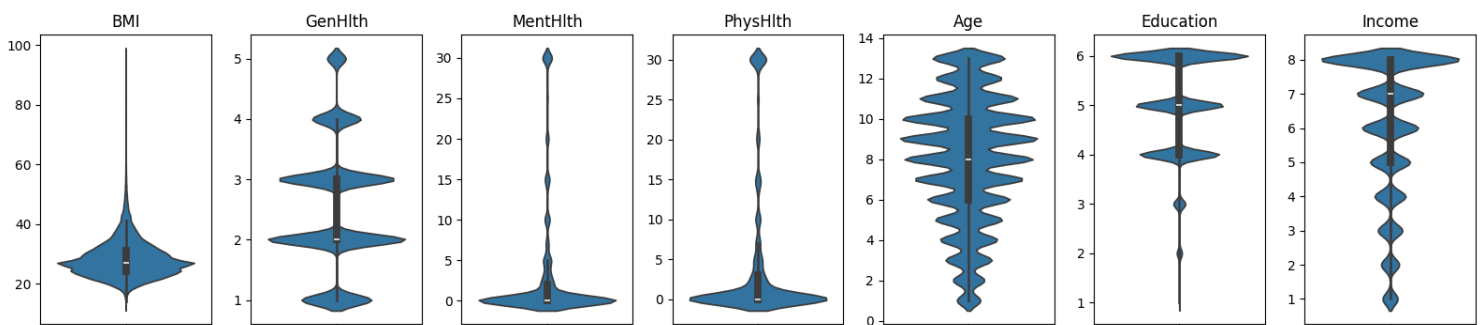
['BMI', 'GenHlth', 'MentHlth', 'PhysHlth', 'Age', 'Education', 'Income']

These variables represent a mix of data types:

- **Pure continuous:** e.g., **BMI**, which reflects actual body mass index values.
- **Binned numerical features:** e.g. **Age**, which has been grouped into intervals.
- **Ordinal categorical (numeric-coded):** e.g. **GenHlth**, which captures general health on a ranked scale (1-5).

Visualization: Violin Plot

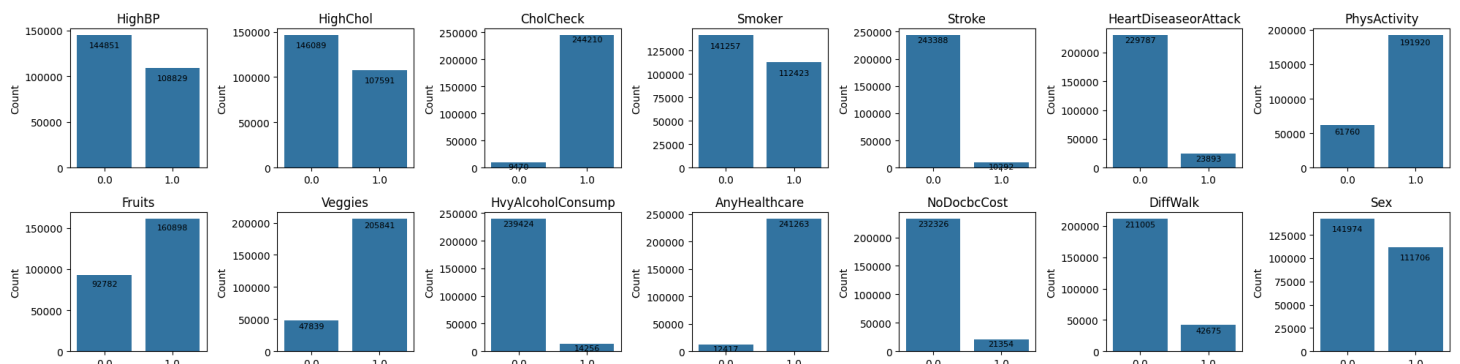
Distribution of Continuous Features (Violin Plots)



A violin plot was generated to visualize the distribution of continuous and ordinal features. This plot illustrates the presence of different data types—showing smooth, continuous distributions for features like **BMI**, discrete bins for **Age**, and categorical-like peaks for ordinal variables such as **GenHlth**.

Visualization: Histogram of Binary Features

Distribution of Binary Features (Count Plots)



Histograms were used to inspect the distribution of binary variables. The plots reveal imbalanced distributions across several features—for example, significantly more individuals reported **No** than **Yes** for Stroke or **HeartDiseaseorAttack**.

Target Variable Distribution

Initial Imbalance Observations

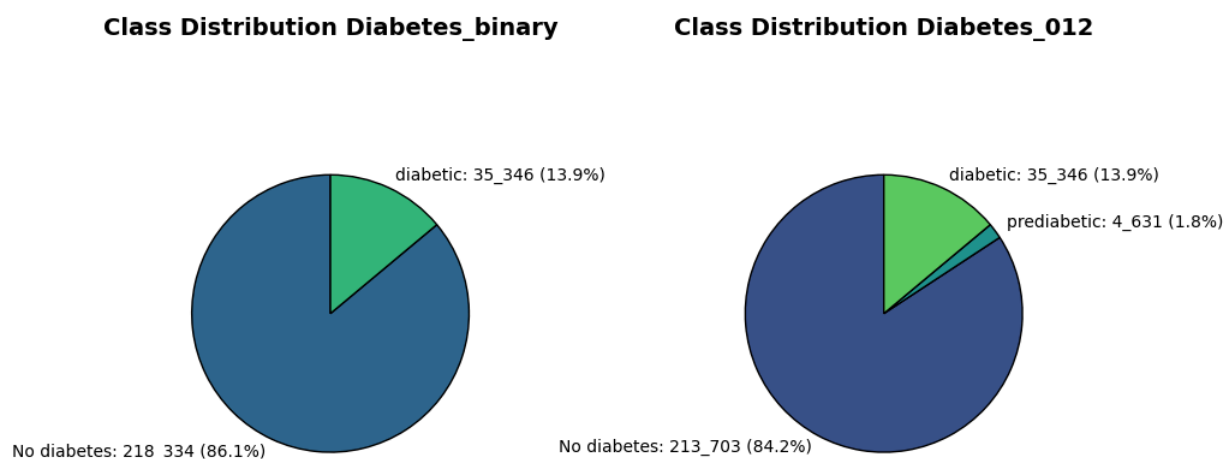
An important insight from both the violin and histogram plots is the presence of **class imbalance**:

- In the violin plots, most continuous and ordinal features show skewed distributions toward certain classes.
- For binary features, a strong disparity is evident, with one class typically dominating the other in frequency.

This imbalance will be an important consideration for preprocessing and model evaluation.

Class Imbalance Assessment

To evaluate potential class imbalance in the target variables, pie charts were generated for both **Diabetes_binary** and **Diabetes_012**.



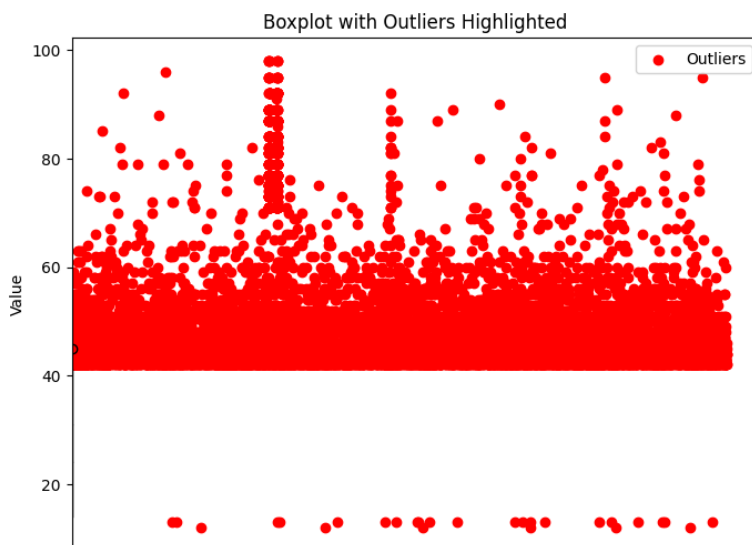
The results show a **strong imbalance** in both classification schemes:

- In the **binary classification**, over **86%** of individuals fall under the "No Diabetes" category.
- In the **ternary classification**, the imbalance is even more pronounced:
 - "No Diabetes" accounts for **over 84%** of the samples.
 - The "Prediabetes" class is **extremely underrepresented**, comprising only **1.8%** of the data.

This imbalance has important implications for model training and evaluation. It will be specifically addressed during the **data preprocessing** phase.

Outlier Detection

Boxplot Analysis of BMI



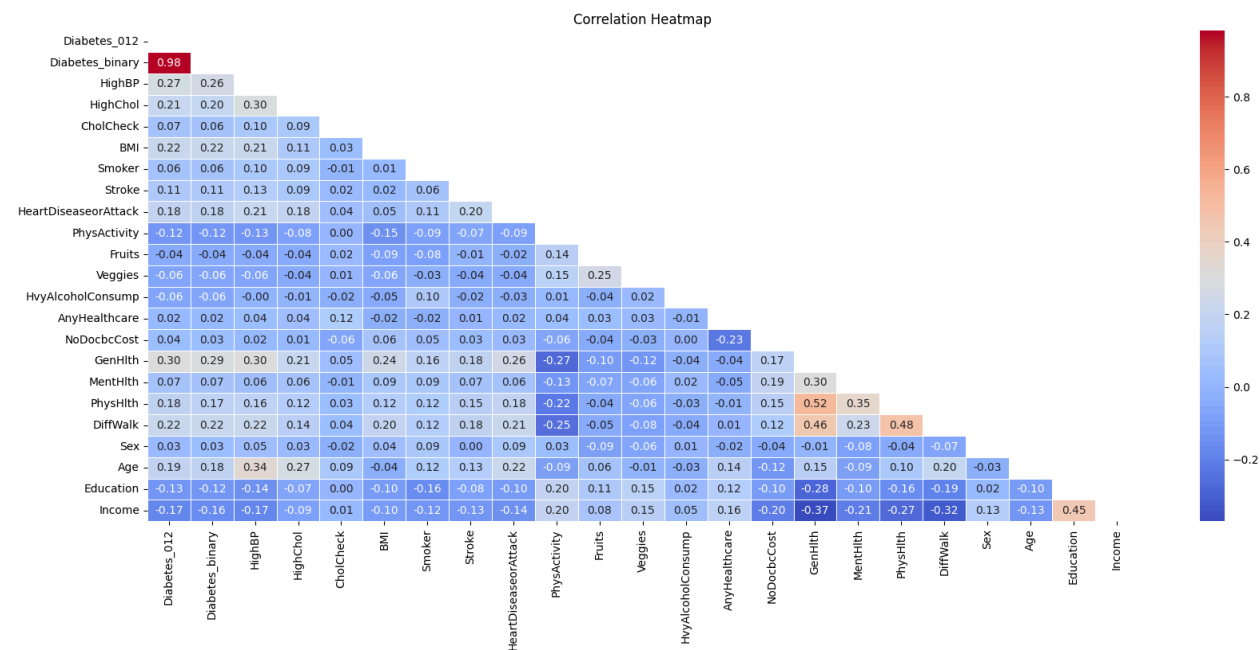
A boxplot of the **BMI** variable was used to identify potential outliers using the **Interquartile Range (IQR) method**.

Although many points fall outside the upper and lower bounds and are flagged as outliers (highlighted in red), these values appear to be **plausible** and **scientifically valid**.

Given the **subjective and self-reported nature** of the BRFSS data and the absence of obviously erroneous values (e.g., biologically impossible BMI scores), **no outliers were removed** from the dataset.

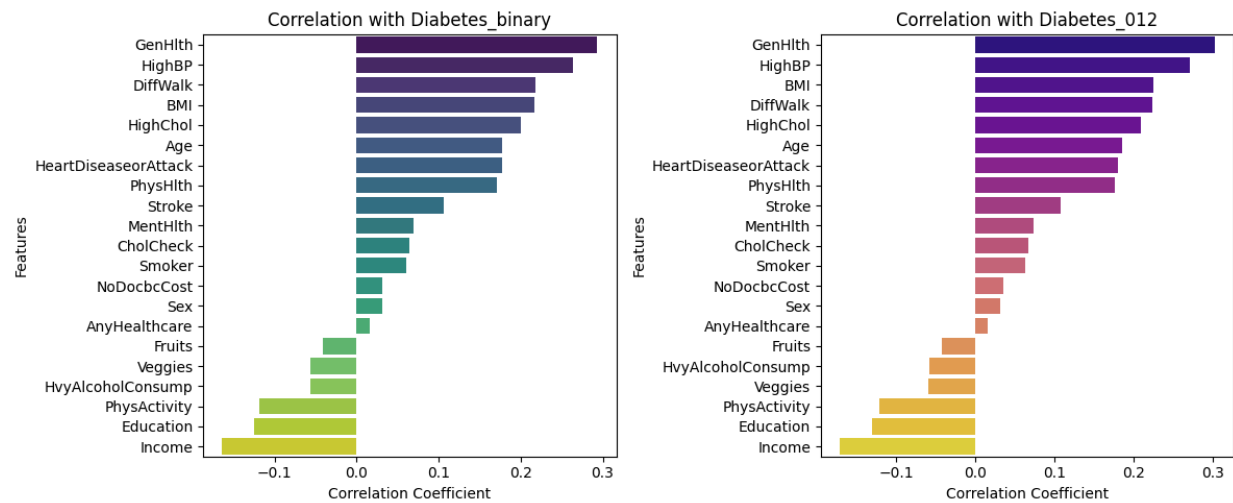
Correlation Analysis

Input Feature Correlation



The analysis reveals **low pairwise correlation** among the features, indicating that most variables are capturing distinct information. As a result, **no features were excluded** due to redundancy.

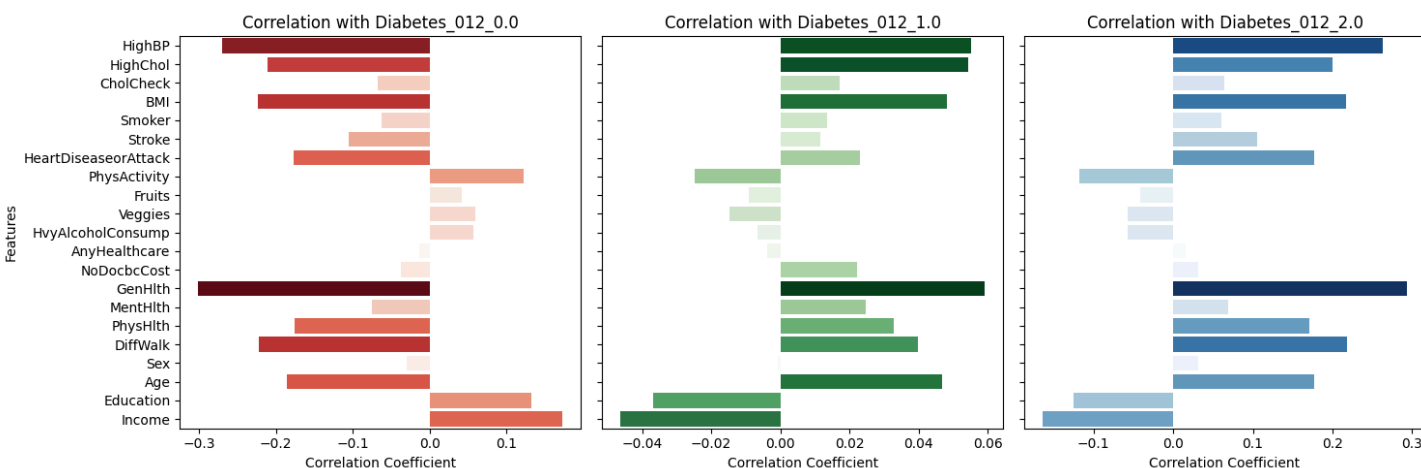
Correlation with Target Variables



Separate correlation heatmaps were created for both **Diabetes_binary** and **Diabetes_012**.

While overall correlation levels are low, the patterns are **nearly identical for both target variables**, which is expected given that the underlying predictors are the same.

Class-Specific Correlation in Ternary Target

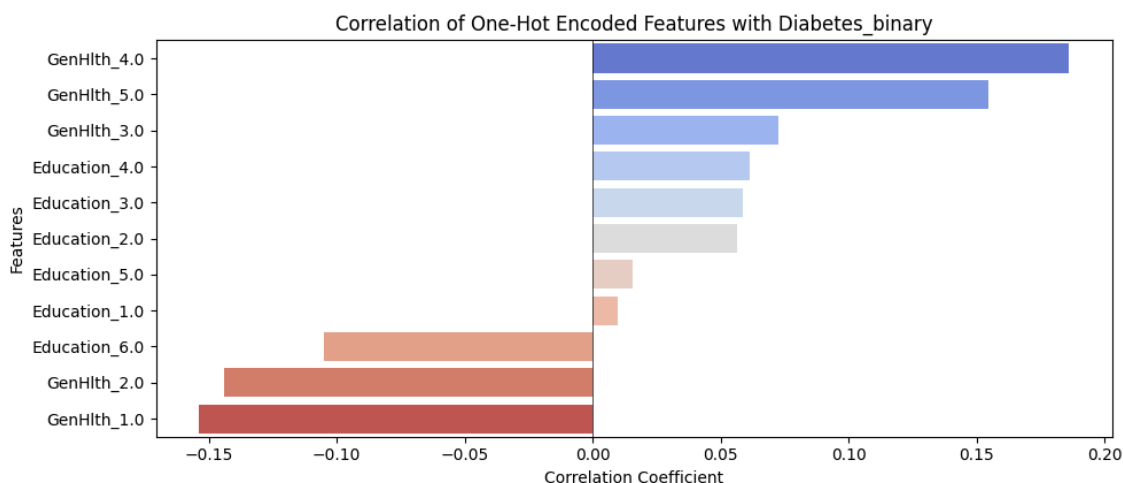


A detailed correlation analysis was conducted between input features and each individual class in the ternary classification target (**Diabetes_012**):

- The **Prediabetes (1)** and **Diabetes (2)** classes exhibit **very similar correlation patterns** across features.
- The **No Diabetes (0)** class shows **opposite correlation trends**, suggesting that its profile differs significantly from the other two.

One-Hot Encoded Feature Correlation with Ternary Target

To further investigate the predictive strength of individual feature levels, one-hot encoding was applied to selected ordinal features, and correlations were computed against the ternary target variable (**Diabetes_012**).

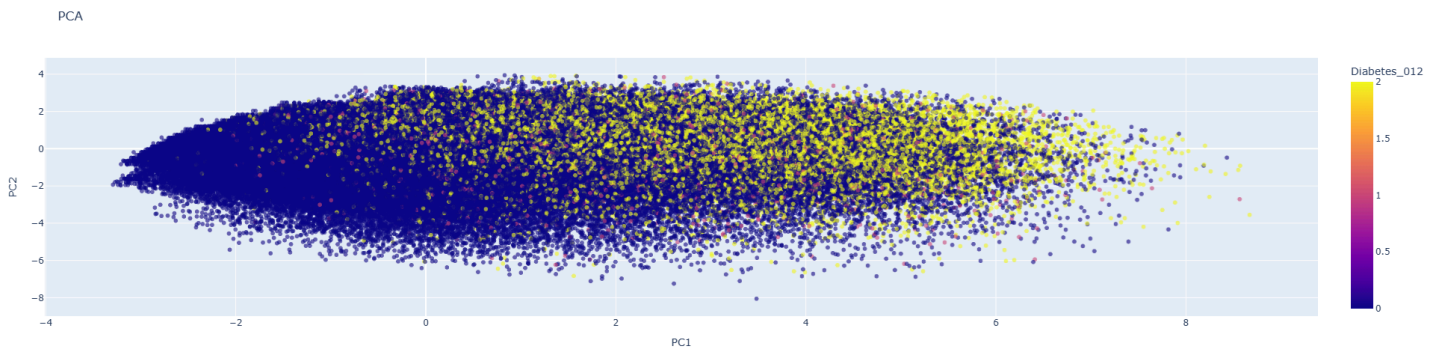


Findings

- The one-hot encoded levels of **GenH1th** demonstrated the **highest correlation** with the target variable, although still below **20%**.
- It is worth noticing that **GenH1th is the most informative single feature** in distinguishing between diabetes classes within this dataset, but despite applying one-hot encoding, we didn't observe an increased percentage in its correlation.
- Due to the **large number of features** and the **low correlation values** observed, **one-hot encoding will not be included** in the final preprocessing pipeline

Principal Component Analysis (PCA)

Principal Component Analysis (PCA) was performed on the **ternary classification dataset**, excluding the target variable, to explore the potential for dimensionality reduction and to visualize class separability.



The PCA projection reveals **some visual separation** between class **0** (No Diabetes) and class **2** (Diabetes), suggesting that PCA may help in identifying diabetic individuals.

Class **1** (Prediabetes) appears **poorly separated**, overlapping significantly with both other classes. This aligns with earlier observations that the **Prediabetes class is underrepresented** and not easily distinguishable based on the input features.

A reevaluation will be conducted **after preprocessing**, including potential resampling, to determine whether class separation improves in reduced-dimensional space.

Data Preprocessing

Data Cleaning

Duplicate Removal

Duplicate records were removed to ensure data consistency and avoid inflating the importance of repeated entries. Duplicated values were more abundant **No Diabetes** class, so it didn't affect the minority classes much. Helping to reduce the disparity between classes.

Outlier and Missing Data Handling

Outlier analysis (e.g., BMI) was previously conducted during the Exploratory Data Analysis phase. No biologically implausible or scientifically invalid values were found. Therefore, **no outliers were removed**.

The dataset **does not contain missing values**, eliminating the need for imputation methods or row removal.

Splitting into Scenarios

- The dataset contains two distinct classification targets:
 - **Diabetes_binary** for binary classification (No Diabetes vs. Diabetes)
 - **Diabetes_012** for ternary classification (No Diabetes, Prediabetes, Diabetes)
- To preserve the **class distributions** unique to each scenario, the dataset was **split into two separate copies**, one for each problem definition.

Data Splitting

Each problem-specific dataset was split into three subsets using **train_test_split**:

- **Training Set (80%)**
- **Validation Set (10%)**
- **Test Set (10%)**

Feature Engineering

Feature engineering was conducted to transform the raw input variables into forms that enhance model performance and are better suited to the assumptions of various algorithms. The following techniques were considered and selectively applied:

One-Hot Encoding (OHE) — *Not Applied*

- One-hot encoding is typically used for nominal categorical variables, which are not present in this dataset. While OHE followed by scaling can be beneficial for certain models (e.g., neural networks), its application was deemed unnecessary here.
- Features in the dataset are either numerical, binary, or ordinal, and are better handled through other preprocessing methods.

Box-Cox Transformation (Normalization)

- **Purpose:** Box-Cox is a power transformation used to approximate a normal distribution for skewed numerical features. It is particularly effective for algorithms that assume input features follow a Gaussian-like distribution.
- **Application Details:** Applied only to **numerical features** with visibly skewed distributions. Performed **prior to scaling**, as standardization assumes a roughly symmetric distribution. As Box-Cox requires strictly **positive values**, features with zero or negative values were either adjusted or excluded from this step. Although **Yeo-Johnson** was a valid alternative that handles non-positive data, it was not used in this pipeline.

Standardization (StandardScaler)

- **Purpose:** Standardization transforms features to have **zero mean and unit variance**, ensuring uniform feature contribution during model training.
- **Implementation:** Applied to **all features** (follows the Box-Cox transformation for numerical features). Ensures consistency across features for algorithms sensitive to feature scaling (e.g., SVM, neural networks, PCA).

Dimensionality Reduction — *Not Applied*

- **Rationale:** Dimensionality reduction techniques, such as PCA, are generally applied **after scaling** and prior to model training to reduce noise or redundancy. While PCA was explored during exploratory analysis for visualization purposes, it was **not integrated** into the final feature engineering pipeline due to limited gains in interpretability or model performance.

Binning — *Not Considered*

Methodologies

The methodologies employed were Random Forests, SVM (Support Vector Machines), and MLP (Multilayer Perceptron, a neural network). Specifically, using the sklearn library for all models, compatible with the grid search methods.

Random Forest

Parameters

n_estimators:

- Number of trees in the forest;
- More trees generally improve performance but increase computation time.

max_depth:

- Maximum depth of each tree;
- Controls how detailed the decision paths can become.

min_samples_split:

- Minimum number of samples required to split an internal node;
- Higher values prevent overfitting.

min_samples_leaf:

- Minimum number of samples required at each leaf node;
- Ensures predictions are based on sufficient data.

Chosen grid search parameters

```
rf_param_grid = {  
    'n_estimators': [50, 100, 200],  
    'max_depth': [None, 10, 20],  
    'min_samples_split': [5, 10],  
    'min_samples_leaf': [2, 4]  
}
```


Support Vector Machines

Parameters

C:

- Regularization parameter controlling the trade-off between margin and classification error.
- Smaller values prioritize larger margins (potential underfitting).
- Larger values focus on correct classification (risk overfitting).

kernel: (not used because we decided on using linear SVM)

- Transforms data into higher-dimensional space to handle non-linear relationships.
- Different kernels suit different data patterns; options include 'linear', 'rbf', and 'poly'.

gamma: (not used because we decided on using linear SVM)

- kernel coefficient defining the influence of individual training examples.
- Low values create smoother boundaries with broader influence;
- High values create more complex boundaries (risk overfitting).

degree: (not used because we decided on using linear SVM)

- Determines the degree of polynomial function for data transformation.
- Only applies to 'poly' kernel; higher degrees can lead to overfitting and increased computational cost.

tol:

- Tolerance for stopping criteria to terminate the optimization algorithm.
- Lower values ensure more precise convergence but require more computation time; higher values may stop prematurely.

max_iter:

- Maximum number of iterations allowed to converge to a solution.
- Higher values give time for convergence on complex problems but increase computation time; too low can result in underfitting.

class_weight:

- Weights inversely proportional to class frequencies.
- 'balanced' handles data imbalance, giving more weight to minority classes.

Chosen grid search parameters

```
svm_param_grid = {  
    'C': uniform(0.01, 100),  
    'tol': [1e-4, 1e-3, 1e-2],  
    'max_iter': [1000, 5000, 10000],  
    'class_weight': ['balanced']  
}
```

Multilayer Perceptron

Parameters

hidden_layer_sizes:

- Defines the architecture of hidden layers in the neural network.
Each tuple defines the number of neurons in consecutive layers.

activation:

- The activation function used in hidden layers to introduce non-linearity.
- 'relu' allows for effective gradient propagation and sparse activation.

solver:

- An adaptive optimization method.
- 'adam' generally works well for most problems.

learning_rate:

- Determines how the learning rate changes during training.
- 'constant' keeps the learning rate fixed throughout the training process.

learning_rate_init:

- The initial learning rate for weight updates at the beginning of training.
- Controls step size in optimization;
- Smaller values provide more precision but slower convergence.

alpha:

- L2 regularization parameter that adds penalty for large weights.
- Higher values increase regularization strength to prevent overfitting.

max_iter:

- Maximum number of training epochs (passes through the dataset).
- Sets the limit on training duration to prevent excessive computation time.

early_stopping:

- Enables termination of training when validation performance stops improving.
- Helps prevent overfitting by stopping before the model memorizes training data.

validation_fraction:

- Portion of training data reserved for validation when early stopping is enabled.

batch_size:

- Number of samples used in each gradient update.
- 'auto' lets the algorithm determine optimal batch size.

random_state: (Fixed at 42 for consistent outcomes)

- Seed value for random number generation to ensure reproducible results.

Chosen grid search parameters

```
nn_mlp_param_grid = {  
    'hidden_layer_sizes': [(64, 32), (30, 15)],  
    'activation': ['relu'],  
    'solver': ['adam'],  
    'learning_rate': ['constant'],  
    'learning_rate_init': [0.001, 0.01],  
    'alpha': [0.0001, 0.01],  
    'max_iter': [1000],  
    'early_stopping': [True],  
    'validation_fraction': [0.1],  
    'batch_size': ['auto'],  
    'random_state': [42]  
}
```

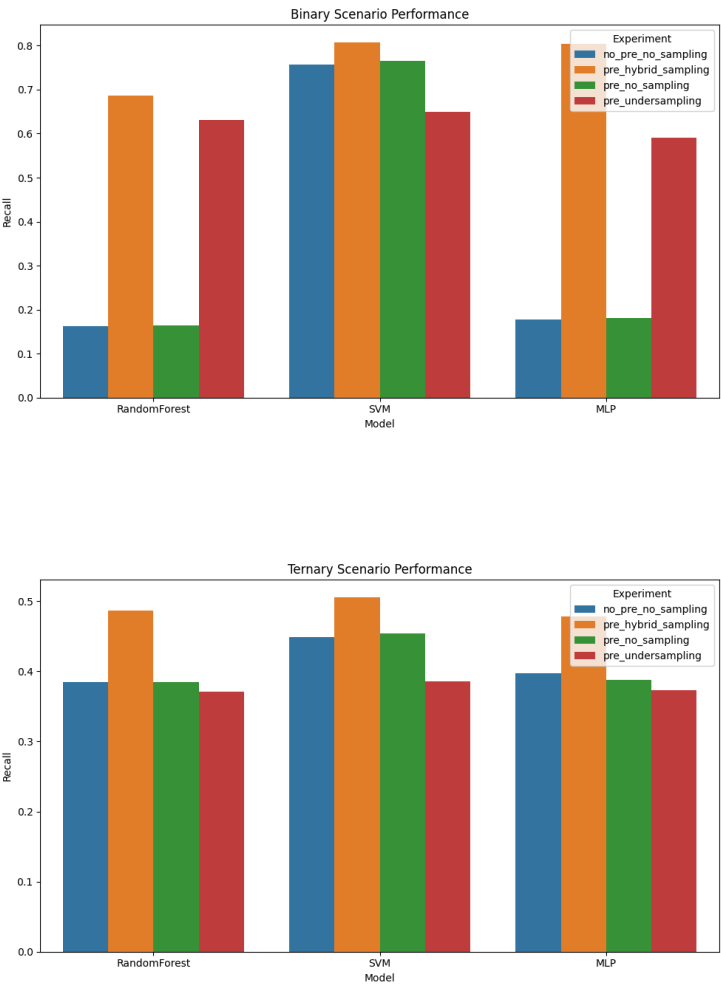
Results and Discussion

The objective of this study includes evaluating the impact of preprocessing techniques and data sampling strategies on the Recall performance of three distinct machine learning models: Random Forest, Support Vector Machine (SVM), and Multi-Layer Perceptron (MLP).

The experiments were structured across four different conditions, systematically varying the application of preprocessing and data sampling: (1) no preprocessing and no sampling (**no_pre_no_sampling**), (2) preprocessing without sampling (**pre_no_sampling**), (3) preprocessing with undersampling (**pre_undersampling**), and (4) preprocessing with a hybrid sampling approach (**pre_hybrid_sampling**).

The Recall metric, crucial for the imbalanced dataset, being paramount, serves as the primary evaluation criterion.

Results



Binary			
Model	Experiment type	Recall	F1
Random Forest	no_pre_no_sampling	0.1624	0.2514
Random Forest	pre_no_sampling	0.1638	0.2525
Random Forest	pre_undersampling	0.6311	0.3204
Random Forest	pre_hybrid_sampling	0.6869	0.4622
SVM	no_pre_no_sampling	0.7567	0.4452
SVM	pre_no_sampling	0.7658	0.4464
SVM	pre_undersampling	0.6490	0.3881
SVM	pre_hybrid_sampling	0.8068	0.4383
MLP	no_pre_no_sampling	0.1772	0.2701
MLP	pre_no_sampling	0.1801	0.2733
MLP	pre_undersampling	0.5900	0.3101
MLP	pre_hybrid_sampling	0.8037	0.4212

Ternary			
Model	Experiment type	recall_macro	f1_weighted
Random Forest	no_pre_no_sampling	0.3846	0.7915
Random Forest	pre_no_sampling	0.3842	0.7910
Random Forest	pre_undersampling	0.3709	0.3900
Random Forest	pre_hybrid_sampling	0.4864	0.7513
SVM	no_pre_no_sampling	0.4483	0.7987
SVM	pre_no_sampling	0.4541	0.8001
SVM	pre_undersampling	0.3857	0.3578
SVM	pre_hybrid_sampling	0.5052	0.6788
MLP	no_pre_no_sampling	0.3969	0.7992
MLP	pre_no_sampling	0.3882	0.7946
MLP	pre_undersampling	0.3731	0.4506
MLP	pre_hybrid_sampling	0.4787	0.645

Analyzing the Binary Scenario

Impact of Preprocessing

Comparing the **no_pre_no_sampling** and **pre_no_sampling** conditions allows for impact assessment of preprocessing alone. Preprocessing demonstrated a marginal positive effect on Recall. Its impact on Recall for these models in the absence of sampling techniques is limited.

Influence of Data Sampling Techniques

Data sampling techniques, particularly hybrid sampling, appears to have a more substantial impact on Recall performance across all models.

Undersampling

The **pre_undersampling** condition yields mixed results. For Random Forest, a significant increase in Recall is observed (0.1638 to 0.6311), suggesting that balancing the class distribution through undersampling is crucial for this model to effectively identify positive instances. Similarly, MLP shows a considerable improvement in Recall under this condition (0.1801 to 0.59 in the binary scenario).

However, the SVM model experiences a decrease in Recall when undersampling is applied after preprocessing (0.7658 to 0.649). This suggests that the aggressive reduction of the majority class through undersampling might lead to the loss of valuable information, negatively impacting its ability to generalize and recall positive instances.

Hybrid Sampling

The **pre_hybrid_sampling** condition generally results in the highest Recall values for all three models. Random Forest achieves a Recall of 0.6869, MLP reaches 0.8037, and SVM attains the highest Recall of 0.8068 under this experimental setup.

The superior performance under hybrid sampling indicates that a balanced approach to handling class imbalance, leveraging the strengths of both oversampling and undersampling techniques in conjunction with preprocessing, is highly effective in improving the models' ability to correctly identify positive instances without significantly compromising the overall dataset information.

Comparative Model Performance

Across the experiments, the SVM model consistently demonstrates the highest Recall values. This suggests that, for this specific dataset and task, SVM is inherently better at recalling positive instances compared to Random Forest and MLP.

Analyzing the Ternary Scenario

In the ternary classification scenario, preprocessing alone shows minimal impact on recall_macro scores across all models. Unlike the binary case, undersampling after preprocessing actually degrades performance for all models, suggesting information critical for multi-class distinction is lost. Hybrid sampling delivers the highest recall_macro values consistently (RF: 0.4864, SVM: 0.5052, MLP: 0.4787), with SVM maintaining superior performance across all conditions, confirming its effectiveness with complex decision boundaries in multi-class problems.

Trade-offs Between Metrics

While optimizing for recall, we observe important performance trade-offs. In the binary scenario, hybrid sampling maximizes recall but doesn't always produce the best F1 scores—particularly for SVM, which achieves its highest F1 (0.4464) with preprocessing alone despite better recall (0.8068) with hybrid sampling. This trade-off becomes more pronounced in the ternary scenario, where the highest f1_weighted scores occur with minimal preprocessing, indicating that sampling techniques may enhance minority class detection at the expense of overall classification accuracy.

Cross-Scenario Pattern Analysis

Consistent patterns across both scenarios include: minimal impact from preprocessing alone, superior results from hybrid sampling, and SVM's generally stronger recall performance. Key divergences include: undersampling's positive effect in binary scenarios but negative impact in ternary cases, more dramatic hybrid sampling improvements in binary classification, and more severe recall/F1 trade-offs in the ternary scenario. These findings suggest that as classification complexity increases, maintaining balanced performance while optimizing recall becomes increasingly challenging.

Optimizing Grid Search - SVM Execution Time

In the SVM model, some major decisions were made due to the time the models took to run, limiting parameters to try and using a randomized gridsearch. This meant we most likely didn't find the best model because it searched very few possible parameters. It did implement a uniform search to finetune the C parameters which was probably crucial to its good performance. In order to try more possible parameters, a more heuristic grid search approach should be used.

Conclusions

The experimental results underscore the significant impact of both preprocessing and data sampling techniques on the Recall performance of machine learning models when dealing with potentially imbalanced datasets. Preprocessing appears to offer modest benefits, particularly for the SVM model. Data sampling, especially hybrid sampling, emerges as a critical strategy for enhancing Recall across all evaluated models.

The findings suggest that the optimal combination of preprocessing and data sampling is model-dependent. While hybrid sampling demonstrates the most promising results overall, the decrease in SVM's performance under undersampling serves as a cautionary note against the indiscriminate application of balancing techniques. Future work could explore more sophisticated sampling methods and hyperparameter tuning to further.

References

- [1] - Alexteboul. (2020). *Diabetes health indicators dataset* [Data set]. Kaggle. <https://www.kaggle.com/datasets/alexteboul/diabetes-health-indicators-dataset>
- [2] - Centers for Disease Control and Prevention. (2018). National diabetes statistics report, 2018. <https://www.cdc.gov/diabetes/php/data-research/index.html>
- [3] - Choi, B. C. K., Shi, L., & Chen, Y. (2019). Building risk prediction models for type 2 diabetes using machine learning techniques. *Preventing Chronic Disease*, 16, E130.
- [4] - Sadhu, A., & Jadli, A. (2022). A survey on diabetes risk prediction using machine learning approaches. *Journal of Medical Systems*, 46(12), 1–12.
- [5] - Zhang, W., Li, J., & Chen, H. (2023). Diabetes risk prediction model based on community follow-up data using machine learning. *Journal of Diabetes and its Complications*, 37(5), 108456.
- [6] - Farhad, M., & Homayounpour, M. M. (2024). Predicting diabetes in adults: Identifying important features in unbalanced data over a 5-year cohort study using machine learning algorithm. *BMC Medical Research Methodology*, 24(1), 213.
- [7] - Nguyen, B. P., Pham, H. N., & Tran, Q. (2021). Machine learning and deep learning predictive models for type 2 diabetes: A systematic review. *Diabetology & Metabolic Syndrome*, 13(1), 148.

Disclaimer: AI tools, including Large Language Models (LLMs) such as ChatGPT (chatgpt.com) and Gemini (gemini.google.com), were utilized to elucidate concepts, formulate references, generate illustrative code examples, and clarify specific inquiries. These tools aided in comprehending certain topics discussed within this report to enhance overall understanding.